THE LOST KINGDOM

Production Requirements for a C++ Console-Based Game

1. Project Purpose

The Lost Kingdom is a production-ready, text-based adventure game written in modern C++. Players explore an ancient civilisation through console menus, make choices, solve puzzles, collect items, manage health and resources, and uncover the reason the kingdom disappeared. The application must be more than a working prototype: it should be stable, understandable, testable, accessible through a standard terminal, and straightforward to build and maintain on supported platforms.

2. Core Gameplay Requirements

- **Game flow:** Provide a clear title screen, new game, continue/load game where supported, instructions, settings, and quit options.
- **Exploration:** Include connected locations with descriptions, available actions, meaningful choices, clues, and clear feedback after each action.
- **Inventory:** Allow players to inspect, collect, use, and where appropriate discard items. Prevent duplicate or invalid item operations.
- **Player state:** Track health and any other resources consistently; define what happens at zero health and when the player completes the main objective.
- **Puzzles and progression:** Validate puzzle answers, preserve completed objectives, prevent impossible progression, and provide fair hints or explanations where appropriate.
- **Consequences and endings:** Ensure choices update the game state correctly and that each ending can be reached through a valid playthrough.
- **Save/load:** Support saving and loading progress if included in the release scope. Validate save files, handle missing or corrupted files gracefully, and avoid silently overwriting a valid save.
- **Help:** Explain controls, commands, objective, and any special mechanics in-game.

3. Engineering and Code Quality

- Use a supported modern C++ standard (preferably C++17 or later) and document the compiler and build steps.
- Separate responsibilities into small functions and appropriate classes, such as Player, Item, Location, GameState, SaveManager, and GameManager. Avoid one oversized main() function.
- Keep game content separate from core logic where practical, using structured data or clearly organised content definitions.
- Use standard library containers and RAII; avoid raw owning pointers and undefined behaviour. Initialise variables and make object ownership explicit.
- Use const-correctness, meaningful names, consistent formatting, and comments that explain intent rather than restating obvious code.
- Treat all user input and loaded data as untrusted. Validate menu choices, numeric ranges, text length, file paths, and save data.
- Handle end-of-file, interrupted input, failed file operations, and unexpected input without infinite loops or uncontrolled termination.

4. Console User Experience and Accessibility

- Present readable menus, consistent prompts, clear status summaries, and concise messages explaining errors and outcomes.
- Never rely on colour alone to communicate meaning; ensure text remains readable in common terminal themes and at typical window sizes.
- Support ordinary keyboard input and avoid requiring a specific terminal width, font, or colour capability.
- Provide a safe, explicit quit path and ask for confirmation before discarding unsaved progress.
- Keep the interface responsive: every menu must offer a valid action, a way back where appropriate, and a clear exit route.

5. Reliability, Security, and Data Handling

- The game must not crash, hang, corrupt progress, or enter an unrecoverable state during normal play or invalid-input testing.
- Use predictable save locations and document where save files are stored. Do not write to protected system directories by default.
- Use safe file handling and validate data before applying it to the game state. If a save cannot be loaded, explain the issue and allow the player to start or load another save.
- Do not collect, transmit, or store personal information unless there is a documented and necessary reason. A local console game should work offline unless a feature explicitly requires connectivity.
- Do not expose internal file paths, debug traces, or sensitive environment information in normal player-facing messages.

6. Testing and Release Readiness

- Unit-test important systems such as inventory changes, health updates, puzzle validation, progression rules, and save/load behaviour.
- Run integration tests for a complete playthrough, alternative choices, every ending, game-over conditions, and save/load continuation.
- Test invalid choices, empty input, alphabetic input where a number is expected, very long input, end-of-file, missing saves, corrupted saves, and insufficient file permissions.
- Compile with strong warnings enabled and resolve warnings. Use sanitizers or equivalent diagnostics during development where available.
- Verify builds on each declared supported operating system and compiler. Do not claim platform support unless it has been tested.
- Provide a README with project overview, requirements, build/run instructions, controls, save-file location, known limitations, and troubleshooting.
- Provide a repeatable build process, version number, release notes, and a clean release package containing only required files.
- Define acceptance criteria: a new player can launch the game, understand the objective, complete a full run, save/load if supported, and exit without data loss.

7. Definition of Done

The game is considered production-ready when all agreed gameplay features are implemented; the project builds reproducibly on its documented platforms; critical gameplay paths and save/load features pass tests; invalid input and file errors are handled safely; no known critical or high-severity defects remain; and the README and release package are complete. Any deliberately postponed features or known limitations must be recorded clearly rather than presented as finished functionality.